

os codigos do chatgpt:

```
# =====
# UNIVERSIDADE / ENGENHARIA CIVIL
# SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ISOSTÁTICAS
# -----
# Autor: ChatGPT
# Linguagem: Python
# =====

import math
import matplotlib.pyplot as plt
from matplotlib.cm import ScalarMappable
from matplotlib.colors import Normalize

# =====
# APRESENTAÇÃO
# =====

print("=" * 75)
print(" SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ")
print("=" * 75)

# =====
# DADOS DOS NÓS
# =====

nos = {
    1: (0, 0),
    2: (4, 0),
    3: (8, 0),
    4: (12, 0),
    5: (2, 3),
    6: (6, 3),
    7: (10, 3),
    8: (14, 3)
}

# =====
# BARRAS
# TRELIÇA ISOSTÁTICA
# 13 barras + 3 reações = 16 incógnitas
# =====

barras = [
    (1, 2),
    (2, 3),
    (3, 4),
```

```

(5, 6),
(6, 7),
(7, 8),
(1, 5),
(2, 5),
(2, 6),
(3, 6),
(3, 7),
(4, 7),
(4, 8)
]

# =====
# CARGAS EXTERNAS
# =====

cargas = {
    6: (0, -25),
    7: (0, -35),
    8: (0, -20)
}

# =====
# APOIOS
# =====

apoios = {
    1: ['x', 'y'],
    4: ['y']
}

# =====
# FUNÇÕES
# =====

def comprimento_barra(no1, no2):

    x1, y1 = nos[no1]
    x2, y2 = nos[no2]

    return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

def cossenos(no1, no2):

    x1, y1 = nos[no1]
    x2, y2 = nos[no2]

```

```

L = comprimento_barra(no1, no2)

cosx = (x2 - x1) / L
cosy = (y2 - y1) / L

return cosx, cosy

# =====
# MONTAGEM DAS INCÓGNITAS
# =====

incognitas = []

for i in range(len(barras)):
    incognitas.append(f'F{i+1}')

for no, tipos in apoios.items():

    for t in tipos:
        incognitas.append(f'R{no}{t}')

# =====
# EXIBIR INCÓGNITAS
# =====

print("\nINCÓGNITAS:")

for i, inc in enumerate(incognitas):
    print(f"{i+1:02d} -> {inc}")

# =====
# EQUAÇÕES
# =====

numero_equacoes = len(nos) * 2
numero_incognitas = len(incognitas)

print("\nNúmero de equações :", numero_equacoes)
print("Número de incógnitas:", numero_incognitas)

# =====
# MATRIZES
# =====

A = [[0.0 for _ in range(numero_incognitas)]
      for _ in range(numero_equacoes)]

B = [0.0 for _ in range(numero_equacoes)]

```

```
# =====  
# MONTAGEM DAS EQUAÇÕES  
# =====
```

```
linha = 0
```

```
for no in nos:
```

```
    # -----  
    # EQUAÇÃO EM X  
    # -----
```

```
    for i, barra in enumerate(barras):
```

```
        n1, n2 = barra
```

```
        if no == n1 or no == n2:
```

```
            if no == n1:  
                cosx, cosy = cossenos(n1, n2)  
            else:  
                cosx, cosy = cossenos(n2, n1)
```

```
            A[linha][i] = cosx
```

```
coluna_reacao = len(barras)
```

```
for no_apoio, tipos in apoios.items():
```

```
    for t in tipos:
```

```
        if no == no_apoio and t == 'x':  
            A[linha][coluna_reacao] = 1
```

```
        coluna_reacao += 1
```

```
if no in cargas:
```

```
    fx, fy = cargas[no]  
    B[linha] = -fx
```

```
linha += 1
```

```
    # -----  
    # EQUAÇÃO EM Y  
    # -----
```

```
    for i, barra in enumerate(barras):
```

```

n1, n2 = barra

if no == n1 or no == n2:

    if no == n1:
        cosx, cosy = cossenos(n1, n2)
    else:
        cosx, cosy = cossenos(n2, n1)

    A[linha][i] = cosy

coluna_reacao = len(barras)

for no_apoio, tipos in apoios.items():

    for t in tipos:

        if no == no_apoio and t == 'y':
            A[linha][coluna_reacao] = 1

        coluna_reacao += 1

    if no in cargas:
        fx, fy = cargas[no]
        B[linha] = -fy

    linha += 1

# =====
# ELIMINAÇÃO DE GAUSS-JORDAN
# =====

def gauss_jordan(matriz, vetor):

    n = len(vetor)

    # Matriz aumentada
    M = []

    for i in range(n):
        M.append(matriz[i] + [vetor[i]])

    # Eliminação
    for i in range(n):

        # Procurar maior pivô
        maior_linha = i

```

```

for k in range(i + 1, n):

    if abs(M[k][i]) > abs(M[maior_linha][i]):
        maior_linha = k

# Troca de linhas
M[i], M[maior_linha] = M[maior_linha], M[i]

pivô = M[i][i]

# Verificação
if abs(pivô) < 1e-12:

    raise ValueError(
        "Sistema impossível ou indeterminado."
    )

# Normalização
for j in range(i, n + 1):
    M[i][j] /= pivô

# Eliminar outras linhas
for k in range(n):

    if k != i:

        fator = M[k][i]

        for j in range(i, n + 1):
            M[k][j] -= fator * M[i][j]

# Solução
solucao = []

for i in range(n):
    solucao.append(M[i][n])

return solucao

# =====
# RESOLVER SISTEMA
# =====

solucao = gauss_jordan(A, B)

# =====
# ORGANIZAÇÃO DOS RESULTADOS

```

```

# =====

forcas_barras = {}
reacoes = {}

for i in range(len(barras)):
    forcas_barras[i + 1] = solucao[i]

indice = len(barras)

for no, tipos in apoios.items():

    for t in tipos:

        nome = f'R{no}{t}'
        reacoes[nome] = solucao[indice]

        indice += 1

# =====
# CLASSIFICAÇÃO DAS BARRAS
# =====

classificacao = {}

for barra, valor in forcas_barras.items():

    if abs(valor) < 0.001:
        classificacao[barra] = "BARRA NULA"

    elif valor > 0:
        classificacao[barra] = "TRAÇÃO"

    else:
        classificacao[barra] = "COMPRESSÃO"

# =====
# ESTATÍSTICAS
# =====

valores_absolutos = [abs(v) for v in forcas_barras.values()]

barra_critica = max(
    forcas_barras,
    key=lambda x: abs(forcas_barras[x])
)

forca_media = sum(valores_absolutos) / len(valores_absolutos)

```

```

# Soma das reações
soma_rx = 0
soma_ry = 0

for nome, valor in reacoes.items():

    if 'x' in nome:
        soma_rx += valor

    elif 'y' in nome:
        soma_ry += valor

# =====
# RESULTADOS DAS BARRAS
# =====

print("\n")
print("=" * 90)
print("RESULTADOS DAS BARRAS")
print("=" * 90)

print(f'{"Barra":<10}{"Nós":<15}{"Força (kN)":<20}{"Classificação":<20}')
```

print("-" * 90)

```

for i, barra in enumerate(barras):

    numero = i + 1
    n1, n2 = barra
    valor = forcas_barras[numero]

    print(
        f'{"numero":<10}'
        f'{"{str(n1)}+ '{-}' +{str(n2)}:<15}"
        f'{"valor":<20.3f}'
        f'{"classificacao[numero]:<20}"
    )

# =====
# REAÇÕES
# =====

print("\n")
print("=" * 60)
print("REAÇÕES DE APOIO")
print("=" * 60)

```



```

for nome, valor in reacoes.items():
    print(f"{nome:<10} = {valor:10.3f} kN")

# =====
# ESTATÍSTICAS
# =====

print("\n")
print("=" * 60)
print("ESTATÍSTICAS")
print("=" * 60)

print(f"Barra crítica      : Barra {barra_critica}")
print(f"Força máxima      : {forcas_barras[barra_critica]:.3f} kN")
print(f"Força média       : {forca_media:.3f} kN")
print(f"Soma Rx           : {soma_rx:.3f} kN")
print(f"Soma Ry           : {soma_ry:.3f} kN")

# =====
# DESENHO DOS APOIOS
# =====

def desenhar_apoio_fixo(x, y):

    plt.plot(
        [x - 0.4, x + 0.4],
        [y - 0.4, y - 0.4],
        linewidth=3
    )

    plt.plot(
        [x - 0.4, x],
        [y - 0.4, y],
        linewidth=3
    )

    plt.plot(
        [x + 0.4, x],
        [y - 0.4, y],
        linewidth=3
    )

def desenhar_apoio_movel(x, y):

    plt.plot(
        [x - 0.4, x + 0.4],
        [y - 0.4, y - 0.4],

```

```

        linewidth=3
    )

    plt.plot(
        [x - 0.4, x],
        [y - 0.4, y],
        linewidth=3
    )

    plt.plot(
        [x + 0.4, x],
        [y - 0.4, y],
        linewidth=3
    )

    plt.plot(x - 0.2, y - 0.55, 'o')
    plt.plot(x + 0.2, y - 0.55, 'o')

# =====
# GRÁFICO 1 - TRELIÇA
# =====

plt.figure(figsize=(12, 7))

for barra in barras:

    n1, n2 = barra

    x1, y1 = nos[n1]
    x2, y2 = nos[n2]

    plt.plot([x1, x2], [y1, y2], 'k-', linewidth=2)

# Nós
for no, (x, y) in nos.items():

    plt.plot(x, y, 'bo', markersize=8)

    plt.text(
        x + 0.1,
        y + 0.1,
        f'N{no}',
        fontsize=10,
        weight='bold'
    )

# Apoios
desenhar_apoio_fixo(*nos[1])

```

```

desenhar_apoio_movel(*nos[4])

plt.title("TRELIÇA ORIGINAL", fontsize=16, weight='bold')
plt.xlabel("X (m)")
plt.ylabel("Y (m)")
plt.grid(True)
plt.axis('equal')

# =====
# GRÁFICO 2 - TRELIÇA COLORIDA
# =====

plt.figure(figsize=(13, 8))

valores = list(forcas_barras.values())

norm = Normalize(
    vmin=min(valores),
    vmax=max(valores)
)

mapa = plt.cm.seismic

for i, barra in enumerate(barras):

    n1, n2 = barra

    x1, y1 = nos[n1]
    x2, y2 = nos[n2]

    valor = forcas_barras[i + 1]

    cor = mapa(norm(valor))

    plt.plot(
        [x1, x2],
        [y1, y2],
        color=cor,
        linewidth=4
    )

# Nós
for no, (x, y) in nos.items():

    plt.plot(x, y, 'ko')

    plt.text(
        x + 0.1,

```

```

        y + 0.1,
        str(no),
        fontsize=10
    )

sm = ScalarMappable(cmap=mapa, norm=norm)
sm.set_array([])

barra_cor = plt.colorbar(sm)
barra_cor.set_label("Força Interna (kN)")

plt.title(
    "TRELIÇA COLORIDA POR FORÇA INTERNA",
    fontsize=15,
    weight='bold'
)

plt.xlabel("X (m)")
plt.ylabel("Y (m)")
plt.grid(True)
plt.axis('equal')

# =====
# GRÁFICO 3 - BARRAS
# =====

plt.figure(figsize=(14, 6))

indices = list(forcas_barras.keys())
valores = list(forcas_barras.values())

cores = []

for v in valores:

    if v > 0:
        cores.append('blue')

    elif v < 0:
        cores.append('red')

    else:
        cores.append('gray')

plt.bar(indices, valores, color=cores)

plt.axhline(0, color='black')

```

```

plt.xlabel("Barra")
plt.ylabel("Força (kN)")

plt.title(
    "FORÇAS INTERNAS NAS BARRAS",
    fontsize=15,
    weight='bold'
)

plt.grid(axis='y')

# =====
# HISTOGRAMA
# =====

plt.figure(figsize=(12, 6))

plt.hist(
    valores,
    bins=10,
    edgecolor='black'
)

plt.xlabel("Força Interna (kN)")
plt.ylabel("Frequência")

plt.title(
    "DISTRIBUIÇÃO DAS FORÇAS INTERNAS",
    fontsize=15,
    weight='bold'
)

plt.grid(True)

# =====
# MOSTRAR GRÁFICOS
# =====

plt.show()

# =====
# FINALIZAÇÃO
# =====

print("\n")
print("=" * 75)
print("ANÁLISE FINALIZADA COM SUCESSO")
print("=" * 75)

```

```
print("""  
✓ Reações calculadas  
✓ Forças internas determinadas  
✓ Barras classificadas  
✓ Gráficos gerados  
✓ Estrutura analisada com sucesso  
""")
```